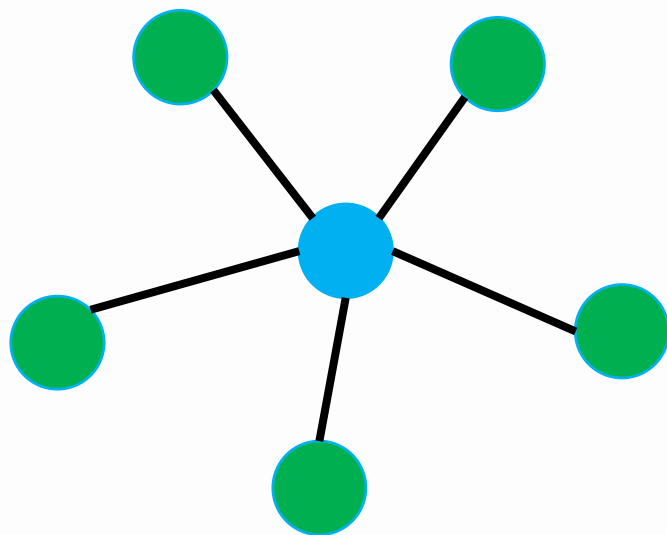


迷宫

假设你此时身处一个很大很大的迷宫中，你什么都不知道，只知道这个迷宫有一个出口，迷宫中每一个格子有很多条路和其他格子相连。而你要做的就是走到这个出口去，假设你有足够的水和食物保证你能生存下来。那么你需要设计一个策略保证你能走出去。



迷宫

首先，如果你在迷宫中到处乱闯，就算你有足够的水和食物，但是你有很大可能陷入一个死循环，一直在打转，永远也走不出去。

因为你对迷宫信息毫不知情，那么如果要保证你一定可以走出迷宫，那么只有一种可能——当你把整个迷宫都走遍了。

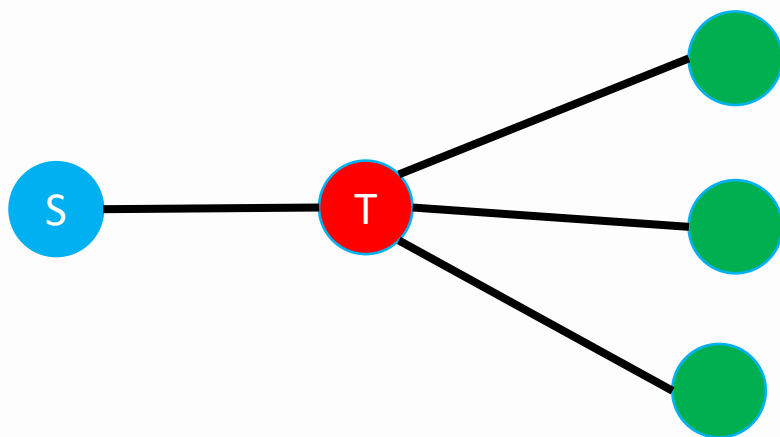
那么，现在你就需要思考：

- (1) 如果我们不想走回头路，应该如何设计策略？
- (2) 为了让你的行程看上去不杂乱，你每次走到下一个位置应该选择什么策略？
- (3) 你怎么才知道迷宫中的哪些位置你走过，哪些位置你没有走过？

迷宫

(1) 如果我们不想走回头路，应该如何设计策略？

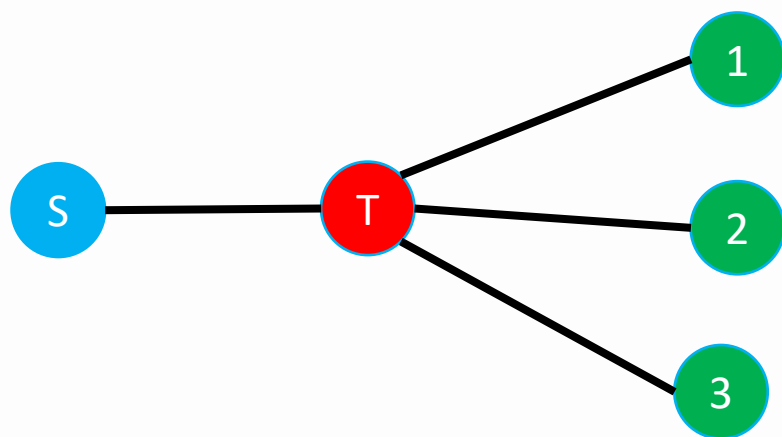
当我们从一个位置走到下一个位置的时候，那么此时我们有两种选择。第一种是回到上一个位置，第二种是继续往下面走。如果我们不想走回头路，那么我们的策略就应该是一直往后面走，直到走到死路位置。



迷宫

(2) 为了让你的行程看上去不杂乱，你每次走到下一个位置应该选择什么策略？

每一个位置面前都有几条路可以选择，为了让行程不杂乱，我们可以按照自己给定点的顺序给这些道路编一个号，每次都按顺序走这些道路。

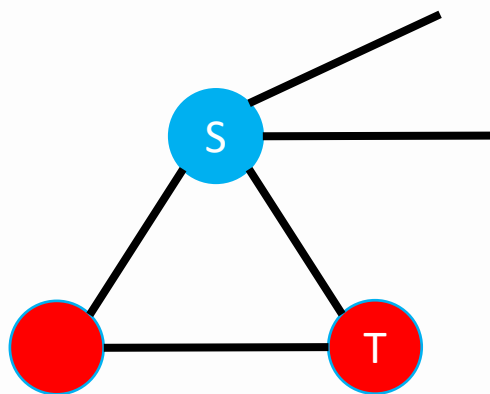


迷宫

(3) 你怎么才知道迷宫中的哪些位置你走过，哪些位置你没有走过？

这个很容易想到，在每一个位置做一个标记，在代码中表现为`book[i]=1`. 如果走到一个位置，这个位置有标记说明我们走过了。

如果这个位置走过了，我们应该怎么办呢？返回上一个位置，换一条路继续往下走。



搜索

前面的这种策略叫做一种搜索策略，我们把这种策略转换成程序来实现。

为了方便，我们可以先把迷宫看做 $n*m$ 的方格，每次只能上下左右走。

那么题意可以转换为：你处于方格中的一个位置，问你是否可以走到某一个特定的位置(出口)

那么：

- (1) 怎么实现行走路线？
- (2) 怎么实现哪些位置被走过？

搜索

对于方格中的任意一点，你可以往上、下、左、右走。

	$(x-1, y)$	
$(x, y-1)$	(x, y)	$(x, y+1)$
	$(x+1, y)$	

我们可以自己定义一个行走顺序，比如上，右，下，左，那么用数字代替就是0, 1, 2, 3

每一个位置的上下左右位置和该点坐标都有关系，我们可以用坐标偏移量来表示，那么从该点出发第0条路的坐标偏移量就是 $(-1, 0)$ ，第1条路就是 $(0, 1)$ ，第2条路就是 $(1, 0)$ ，第3条路就是 $(0, -1)$ 。

我们可以开一个数组来存储 $\text{next}[4][2] = \{-1, 0, 0, 1, 1, 0, 0, -1\}$

搜索

如果从一个点可以走到上下左右，左上，右上，左下，右下八个点，坐标偏移数组应该如何写？

如果从一个点可以走到横坐标翻倍，纵坐标翻倍，横坐标减半，纵坐标减半的四个位置，坐标偏移量应该如何写？

搜索

`next[4][2] = {-1, 0, 0, 1, 1, 0, 0, -1}`

`next[i][0]`: 表示从该点出发第*i*条路的x坐标偏移量

`next[i][1]`: 表示从该点出发第*i*条路的y坐标偏移量

那么如果该点坐标是(x, y), 那么下一个点的坐标就是(x+next[i][0], y+next[i][1])

但是我们要注意, 对于迷宫中最外层的点, 并不是上下左右都可以走的, 因为迷宫是有限大的, 所以我们需要判断一下, 对于x坐标, 如果 $x < 1 \vee x > n$, 对于y坐标, 如果 $y < 1 \vee y > m$, 那么这些点在迷宫中实际上是不存在的, 所以不能走。

搜索

前面我们是直接标记`book[i]=1`就可以了，在标记的时候一定要注意没有歧义，我们没走到一个点，就给这个表标一个递增的号码，那么号码是不会重复的，所以不会有歧义。但是在方格中，无论用行标还是列标来标记都会有歧义，所以我们可以用二维坐标或者按照行一直标号下去 $((x-1)*m+y)$ ，虽然都没有问题，但是我们还要考虑坐标偏移量，所以用二维坐标 (x, y) 好一些，那么标记就变成了`book[x][y]`，表示 (x, y) 这个位置被访问过了。

搜索

前面我们是直接标记`book[i]=1`就可以了，在标记的时候一定要注意没有歧义，我们没走到一个点，就给这个表标一个递增的号码，那么号码是不会重复的，所以不会有歧义。但是在方格中，无论用行标还是列标来标记都会有歧义，所以我们可以用二维坐标或者按照行一直标号下去 $((x-1)*m+y)$ ，虽然都没有问题，但是我们还要考虑坐标偏移量，所以用二维坐标 (x, y) 好一些，那么标记就变成了`book[x][y]`，表示 (x, y) 这个位置被访问过了。

搜索

```
book[x][y]=1;
void dfs(int x,int y)
{
    if(x==X&& y==Y)
        ...
    for(int k=0;k<4;k++)//枚举与x相连的四个点
    {
        int tx=x+nextn[k][0];
        int ty=y+nextn[k][1];
        if(tx<1||tx>n||ty<1||ty>m)
            continue;
        if(!book[tx][ty])
        {
            book[tx][ty]=1;
            dfs(tx,ty);
        }
    }
}
```

搜索

当然，我们都知道对于方阵来说，从任意一个点肯定可以到达任意一个点。所以我们加入障碍物，障碍物表示该位置不可以走。

其他地方都不变，只是可不可以走的地方还需要一个判断语句，为了区分哪些点可以走，哪些点不可以走，我们可以加入一个map数组，`map[x][y]=1`表示该点是障碍物，不可以走，`map[x][y]=0`表示可以走。

```
if(!book[tx][ty]&&!map[tx][ty])
{
    book[tx][ty]=1;
    dfs(tx,ty);
}
```

最短路

前面是深搜的模型，现在很多时候都不是求从一个点是否可以到达另一个点，而是求从一个点到达另一个点的最短路。

按照前面的做法，我们可以得到从起点到终点的一条路径，但是这条路径是否是最短的路径我们是不知道的，我们要如何才能知道我们得到的一定是从起点到终点的最短路径呢？

当我们把从起点到终点所有的可能路径求出来之后，其中最短的那一条就是最短路径。

当我们走到终点的时候，表示我们已经得到了一条长度为len的路径，但是可能还有其他路径更短，所以我们需要先退回上一步，再重新尝试其他可能。也就是撤销操作，所以我们要取消对该点的标记。

最短路

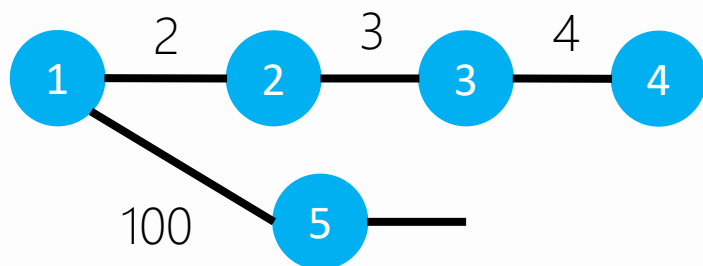
```
book[x][y]=1;
void dfs(int x,int y,int step)
{
    if(x==X&&y==Y)
    {
        if(step<min)
            min=step;
        return;
    }
    for(int k=0;k<4;k++)//枚举与x相连的四个点
    {
        int tx=x+nextn[k][0];
        int ty=y+nextn[k][1];
        if(tx<1||tx>n||ty<1||ty>m)
            continue;
        if(!book[tx][ty]&&!map[tx][ty])
        {
            book[tx][ty]=1;
            dfs(tx,ty,step+1);
            book[tx][ty]=0;
        }
    }
}
```

最短路

这样做方法是没有问题的，但是实践起来会发现效率太低了，因为可能的路径太多了，所以我们需要优化，也就是排除掉不可能是最短路的路径。

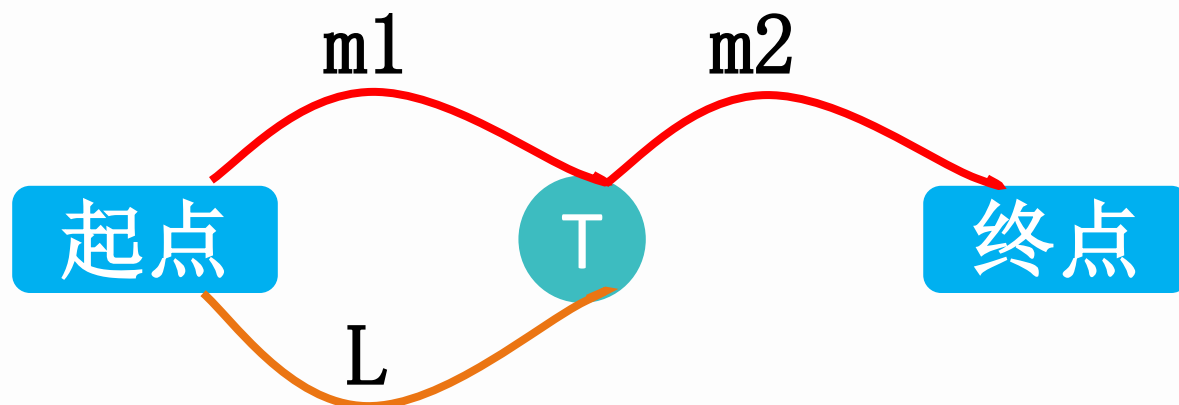
(1) 有重复的路径肯定不是最短路

(2) 前面我们是走到终点再判断和前面求出来的最短路的大小关系，但是实际上因为路径都是正数，所以只要在中途，路径长度 $\geq$ 前面求出来的最短路，那么实际上就没有必要搜索下去了。肯定不是最短路，例如下图，搜索到5号结点就没必要继续搜索下去了。



```
if(step>=Min)    return ;
if(x==X&&y==Y)
{
    min=step;    return;
}
```


最优性剪枝



在已知最优解中，起点到某一点的最短路径为 $m1$ ，如果其他路径中起点到该点的距离 $L > m1$ ，也不需要再往下搜索了。

```
if(step >= dis[x][y])  
    return ;  
else  
    dis[x][y] = step;
```

dis[x][y]: 起点到点(x, y)的当前最短距离。搜到一个点，要么更新，要么回溯。

迷宫

题目描述

有一个仅由数字0与1组成的 $n \times n$ 格迷宫。你可以从一个位置移动到上下左右的另一个位置。但是相同数字间的移动需要花费代价为1，不同数字间的移动需要花费的代价为2。请问怎么从起点移动到终点花费的代价最少。最少代价是多少？

输入

第一行 n ，后面一个 $n \times n$ 的矩阵。最后一行分别是起点和终点的行列坐标

输出

最少代价

样例输入

```
3
0 1 0
0 0 1
1 1 0
1 1 3 3
```

样例输出

```
6
```



迷宫

这道题就是一道最短路的模板题，只是在移动的时候有一点变化，需要分情况讨论，需要判断该点颜色和下一个点颜色的区别，如果相同，那么路径长度+1，不相同，则路径长度+2.



棋盘

题目描述

[展开](#)

有一个 $m \times m$ 的棋盘，棋盘上每一个格子可能是红色、黄色或没有任何颜色的。你现在要从棋盘的最左上角走到棋盘的最右下角。

任何一个时刻，你所站在的位置必须是有颜色的（不能是无色的），你只能向上、下、左、右四个方向前进。当你从一个格子走向另一个格子时，如果两个格子的颜色相同，那你不需要花费金币；如果不同，则你需要花费 1 个金币。

另外，你可以花费 2 个金币施展魔法让下一个无色格子暂时变为你指定的颜色。但这个魔法不能连续使用，而且这个魔法的持续时间很短，也就是说，如果你使用了这个魔法，走到了这个暂时有颜色的格子上，你就不能继续使用魔法；只有当你离开这个位置，走到一个本来就有颜色的格子上的时候，你才能继续使用这个魔法，而当你离开了这个位置（施展魔法使得变为有颜色的格子）时，这个格子恢复为无色。

现在你要从棋盘的最左上角，走到棋盘的最右下角，求花费的最少金币是多少？

棋盘

该题和上一道题比较类似，只是多了一种操作，让一个无色的格子变成任意一种颜色，并且会增加一定的花费，实际上还是最短路的模板题。

解题思路一定要理清楚：

- (1) 上一步是否使用了魔法
- (2) 当前格子实际的颜色
- (3) 如果此次可以使用魔法，那么应该变为什么颜色。
- (4) 我们应该如何判断两个格子颜色是否相同

棋盘

答案是很容易想到的，如果我们需要知道上一步是否使用过魔法，那么我们需要多传递一个参数来表示该次是否使用过魔法。

如果遇到无色的格子，必须要改变颜色，那么肯定变为和当前格子相同颜色最终花费一定比变为和当前格子不同颜色更优

因为无色的格子颜色会被改变，并且只是暂时被改变，一离开就会还原，所以我们改变颜色的时候不能直接改变`map[x][y]`，所以我们可以再传递一个颜色参数。

同样，我们比较的时候，不能直接比较`map[x][y]`和`map[tx][ty]`是否相等，因为可能 (x, y) 的颜色被改变过。

棋盘

理清思路就很简单了。

首先我们先判断可以走到的下一个位置是什么颜色。

如果是无色，再判断是可以用魔法，如果不可以用，那么就走不通了，如果还可以用，那么就用
如果颜色相同并且不是无色，那么直接走就行
如果颜色不同并且不是无色，同样也是直接走。
前面再加一个剪枝就可以过了。

回家

题目描述

展开

小H在一个划分成了 $n*m$ 个方格的长方形封锁线上。每次他能向上下左右四个方向移动一格（当然小H不可以静止不动），但不能离开封锁线，否则就被打死了。刚开始时他有满血6点，每移动一格他要消耗1点血量。一旦小H的血量降到0，他将死去。他可以沿路通过拾取鼠标（什么鬼。。。）来补满血量。只要他走到有鼠标的格子，他不需要任何时间即可拾取。格子上的鼠标可以瞬间补满，所以每次经过这个格子都有鼠标。就算到了某个有鼠标的格子才死去，他也不能通过拾取鼠标补满HP。即使在家门口死去，他也不能算完成任务回到家中。

地图上有5种格子：

数字0：障碍物。

数字1：空地，小H可以自由行走。

数字2：小H出发点，也是一片空地。

数字3：小H的家。

数字4：有鼠标在上面的空地。

小H能否安全回家？如果能，最短需要多长时间呢？

回家

该题也是一道深搜模板题，只是由于题目中的条件，到达任何位置，只要没血了就会死掉，所以每到一个位置要先判断是否有血，再做其他操作。不能先判断是否到家，该位置是否有鼠标，再判断血量。

该题最大的坑点在于，先必须要满足能活下去，再考虑最短路，那么就会有一种可能，为了保命去吃血再返回来回家。比如：

1	1	1	1	1	1	3
0	0	0	0	4	0	0

他必须先到红色位置吃了血再返回来去终点，否则就到不了家，这里的最短路有重复路径。

回家

那么直接标记的做法就是错误的，所以我们要换一种方法。我们先要明白标记的目的是什么？

并不是说不可以走重复的路，而是在以前的题目中，走重复的路，一定不是最优的方案，那么我们就可以直接标记，而在该题中，走重复的路可能是最优的，那么就没必要这么标记了，而不是不标记，不标记肯定死循环。

我们还是要思考出现什么情况才可能一定不是最短路，不再是在是否走过该点，而是上一次到达这个点的血量与后一次到达该点的血量的大小关系。如果下一次到达该点血量比以前少，那么肯定不是最短路。

回家

同样的，既然可以走重复的路，那么前面的剪枝也是有问题的， $\text{map}[x][y]$ ：表示从起点到该点的当前最短路径，很明显如果有重复路径，那么这样判断是错误的，我们应该如何剪枝呢？

只能换回上一种剪枝方法，也就是直接判断当前路径长度和前面最优方案长度的大小。

所以，在做搜索题的时候，一定不能生搬硬套，要注意条件成立的本质。

搜索

在搜索中，最重要的就是求解最短路，我们前面学习的求解最短路的办法无论怎么剪枝就会有重复路径，因为我们的方法都是尝试去走这条路，如果不行再返回，既然是尝试，那么肯定有错误的路径。

我们有没有办法直接让我们走的每一步路都一定是当前的最短路呢？

答案是有的，我们可以按层来搜索，从起点开始，搜索出所有第一层能到达的点，那么从起点到这些点的最短路就知道了，再从这些第一层能到达的所有点依次出发，那么能到达的所有点就是第二层的点，依次类推，那么对于任意一个点，第一次访问到的该点就是起点到该点的最短路径。

搜索

第一层



第二层



第三层



第一种办法就是用数组实现，像上面一样，每一层的点以此放入一个数组中，这样就需要我们开二维数组，第一维表示层数，但是我们不知道最多有多少层，也无法判断最多的一层有多少个点，所以开数组我们都需要开到最大，空间不够，当然，我们发现只有相邻两层之间有关系，那么可以开滚动数组省空间。但是实际上我们发现，先被拓展的点，那么也先拓展其他点，典型的先进先出，所以开一个队列就搞定了。

解题思路

- (1) 把起点放入队列，此时队列里面只有起点一个元素。
- (2) 遍历起点的所有方向，找出所有合理的并且没有被标记过的点放入队列中。
- (3) 当队头元素所有方向遍历完成后，队头元素出队，继续遍历当前队头元素的所有方向并且依次入队。
- (4) 重复执行上述操作，直到队列为空或者找到终点为止。如果队列为空，说明从起点到终点没有路径，如果找到终点，则当前路径长度就是最短路径长度

```
struct Queue  
{  
    int x;  
    int y;  
    int step;  
};  
Queue q[1001000];
```

因为要记录当前位置的行列坐标和最短路径，所以需要用一个结构体

由于该题范围是 $n \leq 1000$. 所以迷宫中最多 1000×1000 个点入队，所以队列必须 $> 1000^2$

最短路径

(1) 起点入队:

```
if (x==X&&y==Y) //特判起点和终点相同
{
    printf("0");
    return 0;
}
int head=1; //定义队头指针
int tail=1; //定义队尾指针指向队尾的下一个位置
q[tail].x=x; //起点入队
q[tail].y=y;
q[tail].step=0;
tail++; //队尾指针后移
book[x][y]=1; //标记起点
```

最短路径

```
while(head<tail) //当队列不为空的时候表示还可以继续搜索
{
```

```
    for(int k=0;k<=3;k++) //分别遍历上下左右四个方向
    {
```

```
        int tx=q[head].x+next[k][0];
        int ty=q[head].y+next[k][1];
        if(tx<1||ty<1||tx>n||ty>n) continue;
        if(map[tx][ty]==0&&book[tx][ty]==0)
        {
```

```
            if(tx==X&&ty==Y)
            {
```

```
                printf("%d", q[head].step+1);
                return 0;
            }
```

```
            book[tx][ty]=1;
```

```
            q[tail].x=tx;    q[tail].y=ty;
```

```
            q[tail].step=q[head].step+1;
```

```
            tail++; //入队之后队尾指针+1
```

如果找到了终点，因为该位置是当前队头位置移动一步到达的，所以用q[head].step+1表示

如果当队列为空时，表示所有可能都搜索完了，还没有找到终点，输出“NO”

```
head++; //当前队头元素所可能遍历完之后，出队
```

```
}
```


STL 队列

定义一个队列:

```
#include<queue>
struct node
{
    int x;
    int y;
    int step;
};
queue<node> q;
```

起点入队:

```
node t; //定义一个结点
t.x=x;
t.y=y;
t.step=0;
q.push(t); //整个结点入栈
```

方法2: 构造函数:

```
struct node
{
    int x;
    int y;
    int step;
    node(x1, y1, step1)
    {
        x=x1;
        y=y1;
        step=step1;
    }
};
queue<node> q;
q.push(node(x, y, 0));
```

不能和上面使用
相同的变量名

STL 队列

```
while(!q.empty())
{
    node tmp=q.front(); //存储队头元素，但是不出队
    for(int k=0;k<=3;k++)
    {
        int tx=tmp.x+next[k][0];
        int ty=tmp.y+next[k][1];
        if(tx<1||tx>n||ty<1||ty>n) continue;
        if(map[tx][ty]==0&&book[tx][ty]==0)
        {
            if(tx==X&&ty==Y)
            {
                printf("%d", tmp.step+1);      return 0;
            }
            q.push(node(tx, ty, tmp.step+1)); //入队
            book[tx][ty]=1;
        }
    }
    q.pop(); //队头元素出队
}
```

搜索

该种搜索方法和前一种深搜方法有本质的区别，前一种搜索方法是尝试每一种可能，从中找到一种最优解，而该种搜索方法每走一步都是到该点的最优解，当走到终点时就是到终点的最优解，所以该种搜索方法几乎没有剪枝方法来提高效率，最多把方格中的所有点出入队一次，时间复杂度是 $O(n*n)$ ，远高于前一种搜索方法。

在深搜中，一般考查搜索剪枝，而在广搜中，主要考查状态唯一性的判断，我们一般用`book[x][y]`来表示在该点的唯一状态，但是很多题中`book[x][y]`表示的位置虽然是唯一的，但是表示的状态却不是唯一的。所以我们不能死记硬背，而是根据实际情况来判断怎么标记状态才唯一并且最优。

POJ3322 Bloxorz

Bloxorz 是一个风靡世界的小游戏。Bloxorz 的地图是一个 N 行 M 列的矩阵，每个位置可能是硬地（用“.”表示）、易碎地面（用“E”表示）、禁地（用“#”表示）、起点（用“X”表示）或终点（用“O”表示）。你的任务是操作一个 $1*1*2$ 的长方体。这个长方体在地面上有两种放置形式，“立”在地面上（ $1*1$ 的面接触地面）或者“躺”在地面上（ $1*2$ 的面接触地面）。在每一步操作中，可以按上下左右四个键之一。按下按键之后，长方体向对应的方向沿着棱滚动 90 度。任意时刻，长方体不能有任何部位接触禁地（否则就会掉下去），并且不能立在易碎地面上（否则会因为压强太大掉下去）。字符“X”标识长方体的起始位置，地图上可能有一个“X”或者两个相邻的“X”。地图上唯一的一个字符“O”标识目标位置。求把长方体移动到目标位置（即立在“O”上）所需要的最少步数。如果无解，输出“Impossible”。在移动过程中，“X”和“O”标识的位置都可以看作是硬地被利用， $3 \leq N, M \leq 500$ 。



P0J3322 Bloxorz

- (1) 这个长方体占的格子不止一格(躺着的时候)，我们该如何表示该长方体所处的位置。
 - (2) 我们该如何标记该位置是否被走过，只用 (x, y) 可以吗？
 - (3) 移动的时候坐标偏移量是什么样子的？
 - (4) 我们该如何判断移动时的边界。
- 

POJ3322 Bloxorz

分析一：该题是求最短路，并且权值为相同，都为一，并且数据范围是300，所以很明显使用广度优先搜索。

分析二：通过画图发现，长方体不仅仅只有两种状态（立着和躺着），而是三种（立着，横躺着，竖躺着），因为横躺和竖躺翻转时坐标偏移量不一样，要让长方体状态唯一，所以我们不但要标记坐标，还需要标记长方体状态0, 1, 2. 所以应该开一个三维标记数组。

分析三：立着的时候，占地面积为1，不能接触禁地和易碎地；横躺着和竖躺着的时候，占地面积为2，都不能接触易碎地。

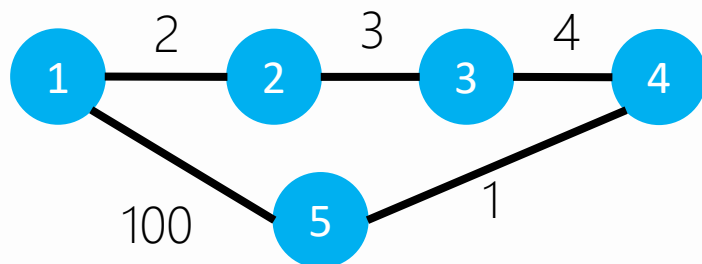
分析四：起点和终点初始状态的判断

P0J3322 Bloxorz

```
if(tt.zt==0)//竖着
{
    if(tt.x<0||tt.y<0||tt.x>=n||tt.y>=m)//超出范围
        continue;
    if(str[tt.x][tt.y]=='E'||str[tt.x][tt.y]=='#')
        continue;
}
else if(tt.zt==1)//横躺着
{
    if(tt.x<0||tt.y<0||tt.x>=n||tt.y>=m-1)
        continue;
    if(str[tt.x][tt.y]=='#'||str[tt.x][tt.y+1]=='#')
        continue;
}
else if(tt.zt==2)
{
    if(tt.x<0||tt.y<0||tt.x>=n-1||tt.y>=m)
        continue;
    if(str[tt.x][tt.y]=='#'||str[tt.x+1][tt.y]=='#')
        continue;
}
```


深搜和广搜

深搜和广搜各有优缺点，从效率上看，深搜的时间复杂度远大于广搜，那么我们为什么还要学习深搜呢？因为广搜是按层拓展，处于每一层的点到起点的距离被认为是相等的，所以每一层的路径长度都必须相等，否则就不可以用广搜，所以深搜的使用范围比广搜大。



深搜的范围不仅仅不如，很多难题不会做的时候，都需要大暴力，但是很多暴力不知道循环的终点，每次循环的变化量都不一样，所以就需要用深搜来打暴力，称为爆搜，所以学好深搜，并且熟练剪枝是很有必要的。

DFS 和 BFS

既然，广度优先搜索的效率远高于深度优先搜索，那么我们为什么还要学习深搜呢？广搜能不能完全代替深搜呢？

我们以前学习的深搜的题包括以下几类：

一：非最短路 (只能用DFS)

- (1) 起点到终点的所有路径
- (2) 起点到终点的所有方案

BFS只能用来求权值相同的最短路问题

二：最短路

- (1) 每走一步花费的代价一样 (DFS和BFS)
- (2) 每走一步花费的代价不一样 (只能用DFS)

综上搜索，因为BFS是层层搜索，所以找到的起点到终点的距离一定是最短路径，并且搜索过一次就不会再搜索第二次了。同样的，当权值不同的时候，层次就会发生错误，第一次求出来的未必就是最短路。



练习题

先自行完成深搜，广搜，双端队列的模板题，没问题之后再完成下面三道题，并在放学之前上交：

- 1: 洛谷 2802 回家
 - 2: 洛谷 3956 棋盘
 - 3: P0J3322 Bloxorz
- 

搜索

有的时候会出现一种奇怪的题，虽然每条路径的长度不完全相同，但是也有规律性，比如路径长度只有0和k(常数)，并且数据范围很大 $n \leq 1000$ ，如果用深搜，时间过不了，如果用广搜，感觉按层拓展又不对。



实际上，当路径长度为0的时候就表示在当前这一层，我们可以很容易用循环来实现，但是实际上我们可以用双端队列来实现。

双端队列

在一个 $n*n$ 的方格中，有两种颜色的格子，在相同颜色之间移动不需要消耗体力，但是在不同颜色之间移动需要消化1个单位的体力，问给定一个起点和一个终点，求起点到终点的最小体力消耗。

$n \leq 2000$

双端队列

这就是一道典型的权值为0和1的搜索题，如果用深搜， $n \leq 2000$ ，肯定超时。如果用普通广搜，搜到一个点就放入队尾，肯定是错误的，因为不满足队列中消耗的递增性。所以我们要用双端队列bfs。如果是相同颜色，无消耗，即和当前队首元素的优先级是一样的，所以我们完全可以把该点放入队首，这样不影响整个队列的有序性。如果是不同的颜色，消耗+1，和普通广搜一样，放入队尾，仍然可以保持原来的优先级。

虽然这种思路很简单，但是在写法细节上和普通广搜还是有一些区别，一定要熟悉两种写法，以免混淆在一起。

搜索

```
,
int head=250000; // 因为要放入队首,
// 所以为了防止数组越界, 队首从n*n开始
int tail=250000;
...
while(head<tail){
    for(int k=0;k<4;k++){
        head++
        ...
        if(map[tx][ty]==map[tmp.x][tmp.y])
        {
            head--; // 注意入队首和队尾的写法
            q[head].x=tx;
            q[head].y=ty;
        }
        else
        {
            q[tail].x=tx;
            q[tail].y=ty;
            tail++;
        }
    }
}
```

搜索

在前面，我们学习的都是如何求解从起点到终点的最短路径长度，但是具体的最短路是哪一条？长度最短，路径不同的最短路有多少条？这些实际上我们从来没有考虑过。

在搜索中，常见的除了求最短路的长度，还经常需要求出最短路的具体某一条路径以及不同的最短路有多少条。

既然常用的搜索方式有深搜和广搜，那么我们就分别来讨论如何用深搜和广搜求最短路径以及最短路的方案数。

深搜求方案数

深搜既然是尝试所有方案的可能性，那么只要能搜到终点的都是最短路，我们完全可以先让其搜到终点之后在判断和当前最短路的大小关系，如果相等，那么方案数+1，如果小了，那么说明前面记录的方案都是错误的，方案数=1.

同样，我们也可以在途中就进行剪枝，但是注意剪枝的时候如果两条路径相同需要继续搜索下去，不能返回。

深搜求方案数

```
void dfs(int x,int y,int step)
```

```
{
```

```
    if(step>Min)//剪枝
```

```
    return ;
```

```
    if(到达终点)
```

```
    {
```

if(step==Min)//假设当前最小值就是最优值，存在一条和他一样优的结果，方案+1
 方案数+1;

else//前面的最小值不是最优值，说明前面的方案都不符合条件，只有当前方案最优
 方案数=1;

```
    }
```

```
}
```

深搜求方案数

```
void dfs(int x, int y, int step)
{
    if(step <= map[x][y]) //相等不能返回
    map[x][y] = step;
    else
    return ;
    if(到达终点)
    {
        if(step == map[x][y])
        方案数+1;
        else
        方案数=1;
    }
}
```

广搜求方案数

广搜，第一次搜到这个点就是到这个点的最短路，但是处于这一层的点路径长度都是相同的，那么也会有其他点可以拓展到终点，比如出于第 $K-1$ 层的点 x 可以拓展到终点，处于第 $K-1$ 层的点 y 可以拓展到终点，处于第 $K-1$ 层的点 z 也可以拓展到终点，那么这三种拓展方案都是最短路的方案数。

那么从起点到终点的最短路径方案数就是3吗？

起点	
	A
B	终点

从起点到终点的最短路径方案数有4种，但是能够到达终点的点明显只有两个点，那么计算出的答案最多也就是2，所以明显是错误的。

那么错误的原因是什么呢？

广搜求方案数

既然从起点到终点的最短路不止一条，那么起点到任意一个点的最短路也很有可能不止一条，但是我们在累加的时候，每个能到达终点的点都是+1，那就意味着起点到x, y, z的最短路无论如何都只有一条，那这明显是不对的。

比如起点到A点的最短路有3条，到B点的最短路有1条，加的时候应该是3+1，而不是1+1，所以我们如果要知道起点到终点的最短路径方案数，还必须知道起点到任意一点的最短路的方案数。

所以重新开一个数组num[x][y]即可。同时还需要一个数组来记录起点到任意一点的层数，层数相同时，累加方案数。

起点	
	A
B	终点

广搜求方案数

```
if(book[tx][ty]==0)
{
    if(tx==X&&ty==Y)//找到了终点
    {
        flag=1;//这一层找完就可以不用找了
    }
    book[tx][ty]=1;
    q.push(node(tx,ty,tmp.step+1));
    num[tx][ty]=num[tmp.x][tmp.y];//继承上一个的方案数
    step[tx][ty]=tmp.step+1;
}
else if(book[tx][ty]==1&&step[tx][ty]==tmp.step+1)
{
    num[tx][ty]+=num[tmp.x][tmp.y];
    //有多个点可以到达，一定要累加
}
```

深搜求具体方案

如果一条路径是从起点到最终点的最短路径，那么这条路径上的任意一个点通过这条路径到起点都一定是最短路径。所以如果我们要知道最短路的具体方案，我们可以通过记录该点由那个点到达或者该点可以到达哪个点来实现。

如果记录该点可以到达哪个点。那么第一个问题是从每一个点出发都可以到达不止一个点，那么就需要记录不止一个点，并且这些点也不一定能到达终点，所以比较麻烦

如果记录该点由哪个点到达，因为我们搜索的时候走的都是当前走过的路径的最短路，所以如果有路径更新，那么就证明最短路更新了，那么到该点的最短路的前驱也就变了，否则不发生改变。而且，到达终点的点一定是从起点出发的，也就是我们记录前驱，从终点开始搜索一定可以到达起点。

当然，路径不止一条，所以一般会限制搜索方式。

深搜求具体方案

```
if(step>=map[x][y].step)
{//前驱不变
    return ;
}
else
{//前驱改变, 更新
    map[x][y].step=step;
    map[x][y].firstx=firstx;
    map[x][y].firsty=firsty;
    if(x==n&&y==m)
    {
        return ;
    }
}
```

```
int k=1;
while(x!=0&&y!=0)
{
    a[k]=x;
    b[k]=y;
    tx=x;
    ty=y;
    x=map[tx][ty].firstx;
    y=map[tx][ty].firsty;
    k++;
}
for(int i=k-1;i>=1;i--)
    printf("%d %d\n",a[i],b[i]);
```



广搜求具体方案

广搜求具体方案数也是一样的原理，广搜中，如果可以到达这个点，那么一定就是最短路径，并且每个点只会被更新一次，所以直接记录前驱就行，根本不需要更新。当更新完终点之后再顺着前驱一直寻找到起点得到的路径就是最终的答案。





练习题

MSOJ 小鼠走迷宫1

MSOJ 小鼠走迷宫2

MSOJ 小鼠走迷宫3



深搜优化

dfs深搜的范围特别广泛，特别是很多难题的暴力算法，但是暴力也分三六九等，虽然暴力不能得满分，但是一个优秀的暴力却可以尽可能多的得分。简单来说，就是爆搜剪枝剪得好，重复状态比较少。

常见的爆搜剪枝方法有下面几种：

- (1) 去除等效冗余
- (2) 估计后期时间
- (3) 二进制优化
- (4) 记忆化搜索
- (5) 优化搜索顺序

深搜优化

(1) 去除等效冗余

如果有几种看似不同的选择效果是完全等价的，我们就可以只选择一种进行尝试。比如**P2386 放苹果**。我们会发现第一个盘子放a个，第二个盘子放b个，和反过来放效果是等价的，我们就不需要考虑了。可以节省很多时间。去除等效冗余一般出现在排列数之中

$$A(n, m) = C(n, m) * A(m, m)$$

深搜优化

(2) 记忆化搜索

记录每个状态的搜索结果，在重复遍历一个状态时直接检索并返回，比如**P1028 数的计算**，就需要记录每个数可以分解的数的个数，当下次分解到该数的时候就直接检索。有的时候还需要记录多个点的状态，这个时候可以考虑状态压缩，以免使用多维数组超出内存限制。

记忆化在很多题中的优化效果是特别好的，甚至等价于动态规划的时间复杂度，实际上动态规划就是记忆化，记录每一种状态，保持不出现重复。

滑雪

题目描述

[展开](#)

Michael喜欢滑雪。这并不奇怪，因为滑雪的确很刺激。可是为了获得速度，滑的区域必须向下倾斜，而且当你滑到坡底，你不得不再次走上坡或者等待升降机来载你。Michael想知道在一个区域中最长的滑坡。区域由一个二维数组给出。数组的每个数字代表点的高度。下面是一个例子：

1	2	3	4	5
16	17	18	19	6
15	24	25	20	7
14	23	22	21	8
13	12	11	10	9

一个人可以从某个点滑向上下左右相邻四个点之一，当且仅当高度减小。在上面的例子中，一条可行的滑坡为24 - 17 - 16 - 1（从24开始，在1结束）。当然25 - 24 - 23 - ... - 3 - 2 - 1更长。事实上，这是最长的一条。

输入格式

输入的第一行为表示区域的二维数组的行数R和列数C（ $1 \leq R, C \leq 100$ ）。下面是R行，每行有C个数，代表高度(两个数字之间用1个空格间隔)。

输出格式

输出区域中最长滑坡的长度。

滑雪

这是一道记忆化搜索的裸题，当我们求出从任意一个点开始滑雪的最远的距离的时候，如果下次我从其他点可以到达该点，那么该点之后的路径实际上已经被求出来了，那么我们就没有必要再去计算一次了，所以就可以直接沿用以前的答案，所以对于每一次的答案，我们开一个数组记录一下最长的路径。

```
int dfs(int x,int y)
{
    if(dis[x][y])//记忆化搜索，如果以前找到了该位置的最长路径就直接返回
        return dis[x][y];
    int flag=0;
    for(int k=0;k<4;k++)
    {
        ...|
        if(map[x][y]>map[tx][ty])
        {
            flag=1;
            dis[x][y]=max(dis[x][y],dfs(tx,ty)+1);//记录该点的最长路径
        }
    }
    ...
    return dis[x][y];
}
```

深搜优化

(3) 优化搜索顺序

在一些搜索问题中，搜索的顺序不一样，搜索的时间也会不同，我们应该分析问题，选择合理的搜索顺序，而不是按部就班的从1搜索到n。比如**POJ 3074 数独游戏**。玩过数独的同学都知道，填数独肯定不是依次填，而是先选择可能性最少的先填，这些效率会高很多。

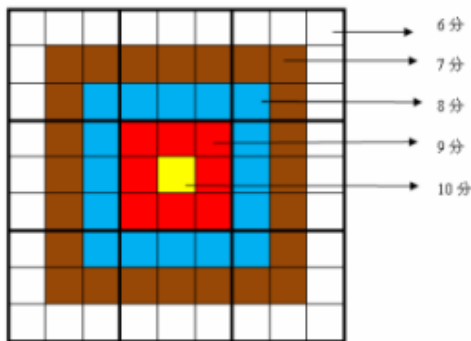
靶形数独

题目描述

展开

小城和小华都是热爱数学的好学生，最近，他们不约而同地迷上了数独游戏，好胜的他们想用数独来一比高低。但普通的数独对他们来说都过于简单了，于是他们向 Z 博士请教，Z 博士拿出了他最近发明的“靶形数独”，作为这两个孩子比试的题目。

靶形数独的方格同普通数独一样，在 9 格宽×9 格高的大九宫格中有 9 个 3 格宽×3 格高的小九宫格（用粗黑色线隔开的）。在这个大九宫格中，有一些数字是已知的，根据这些数字，利用逻辑推理，在其他的空格上填入 1 到 9 的数字。每个数字在每个小九宫格内不能重复出现，每个数字在每行、每列也不能重复出现。但靶形数独有一点和普通数独不同，即每一个方格都有一个分值，而且如同一个靶子一样，离中心越近则分值越高。（如图）



上图具体的分值分布是：最里面一格（黄色区域）为 10 分，黄色区域外面的一圈（红色区域）每个格子为 9 分，再外面一圈（蓝色区域）每个格子为 8 分，蓝色区域外面一圈（棕色区域）每个格子为 7 分，最外面一圈（白色区域）每个格子为 6 分，如上图所示。比赛的要求是：每个人必须完成一个给定的数独（每个给定数独可能有不同的填法），而且要争取更高的总分数。而这个总分数即每个方格上的分值和完成这个数独时填在相应格上的数字的乘积的总和

总分数即每个方格上的分值和完成这个数独时填在相应格上的数字的乘积的总和。如图，在以下的这个已经填完数字的靶形数独游戏中，总分数为 2829。游戏规定，将以总分的高低决出胜负。

7	5	4	9	3	8	2	6	1
1	2	8	6	4	5	9	3	7
6	3	9	2	1	7	4	8	5
8	6	5	4	2	9	1	7	3
9	7	2	3	5	1	6	4	8
4	1	3	8	7	6	5	2	9
5	4	7	1	8	2	3	9	6
2	9	1	7	6	3	8	5	4
3	8	6	5	9	4	7	1	2

由于求胜心切，小城找到了善于编程的你，让你帮他求出，对于给定的靶形数独，能够得到的最高分数。

送礼物

作为惩罚，GY被遣送去帮助某神牛给女生送礼物(GY：貌似是个好差事)但是在GY看到礼物之后，他就不这么认为了。某神牛有N个礼物，且异常沉重，但是GY的力气也异常的大(-_-b)，他一次可以搬动重量和在 w ($w \leq 2^{31}-1$) 以下的任意多个物品。GY希望一次搬掉尽量重的一些物品，请你告诉他在他的力气范围内一次性能搬动的最大重量是多少。

第一行两个整数，分别代表W和N。 ($N \leq 45$)

以后N行，每行一个正整数表示 $G[i]$, $G[i] \leq 2^{31}-1$ 。

仅一个整数，表示GY在他的力气范围内一次性能搬动的最大重量。

送礼物

拿到题一看， n 件物品，拿取其中 k 件质量之和不超 w ，很明显是背包，再看一下数据范围， $2^{31}-1$ ，肯定没法做。如果用搜索呢？ $n \leq 45$ 。一共右 2^{45} 种情况，也超了，但是加上各种优化剪枝说不定可以。但是实际上仍然会超时，我们可以考虑一下 2^{45} 虽然会超，但是 2^{23} 是可以的，所以就可以考虑双向搜索。先dfs前半段，存储所有的可能结果，然后再dfs后判断，对于后面的每一种情况，在前面的情况中快速找到之和 $< w$ 的那组数据，并从中求得最大值。查找很明显可以先排序再用二分查找，时间复杂度为 $2^{23} * \log(2^{23})$ ，会超出一点，剩下的就只能从剪枝中想办法了。

剪枝

一：要使后面选择的物品可能性更小，很明显选择物品的时候从大到小选择。

二：如果刚好选择物品质量= w 直接强行退出程序

三：为了方便后期查找，先排序

四：为了减小搜索规模，序列去重

五：二分查找前半段

剩下的，自己去想，不然仍然会被卡点

剪枝

一：要使后面选择的物品可能性更小，很明显选择物品的时候从大到小选择。

二：如果刚好选择物品质量= w 直接强行退出程序

三：为了方便后期查找，先排序

四：为了减小搜索规模，序列去重

五：二分查找前半段

剩下的，自己去想，不然仍然会被卡点



练习题

1: 洛谷 P2243/P4667 电路维修

2: MS0J 1125 滑雪

3: 洛谷 1074 靶形数独

4: MS0J 1423 送礼物

5: 洛谷 1126 机器人搬重物



深搜进阶题

前面三题作业完成之后，自行练习深搜+各种类型剪枝的题，为以后写爆搜打好基础：

- 1: 洛谷 1205 方块转换
- 2: 洛谷 1092 虫食算
- 3: 洛谷 1731 生日蛋糕
- 4: 洛谷 1120 小木棍